

提高级 CSP-S 第 20 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每题有且仅有一个正确选项)

- 当你在 CSP 第二轮测试的考场上发现自己程序的输出与期望的不一样时,下列做法错误的是()。
 - 浏览一遍自己的程序尝试找出写错的地方
 - 大吵大闹以宣泄自己的情绪
 - 输出并查看中间变量的值是否正确
 - 再读一遍题面确保自己理解题意正确
- NOI 赛场上机试所使用的计算机系统是()。
 - Windows
 - MacOS
 - Linux
 - UNIX
- 在 c++ 中,表达式 $!1 < 20 \sim 23 + 6 * 3 \mid 14$ 等于()。
 - 47
 - 0
 - 14
 - 90
- 以下哪种遍历方式无法确定一棵二叉树()。
 - 先序遍历加中序遍历
 - 中序遍历加后序遍历
 - 先序遍历加后序遍历
 - 先序遍历加中序遍历加后序遍历
- LAN 在计算机科学技术领域的常见含义是()。
 - 蓝
 - 局域网
 - 城域网
 - 广域网
- 以下哪种语言不是面向对象的高级语言()。
 - C++
 - C
 - Java
 - Simula 67
- 令 $f(x)$ 表示 x 的约数个数,求 $f(1) + f(2) + \dots + f(20) = ()$ 。
 - 64
 - 66
 - 68
 - 70
- 当考试中一道题目的空间限制为 512MB 时,如果你的程序除了小部分变量只有一个 double 类型的数组,则这个数组大小的理论上限大概是()。
 - 1.3×10^5
 - 6.5×10^4
 - 1.3×10^8
 - 6.7×10^7
- 以下哪种问题采用动态规划解决所需的时间复杂度最低?()
 - 去同构的子序列数目计数
 - 最长公共子序列
 - 最长上升子序列
 - 最长回文子序列
- 5 个点的无标号无根树的数量为()个。
 - 3
 - 4
 - 5
 - 6
- 以下哪种数据结构无法处理区间翻转操作?()。
 - Splay
 - Treap
 - 无旋 Treap
 - 块状链表
- 在以下的结构体定义下,四个选项中重载运算符的写法正确的是()。


```
struct node{
    int a,b;
};
```

 - istream & operator>>(istream& myin,node& p) {
 cin>>p.a>>p.b;
 return myin;
 }
 - ostream operator<<(ostream& myout,node& p) {
 cout<<p.a<<" "<<p.b<<endl;
 Return myout;
 }
 - bool operator =(node c,node d) {
 return (c.a==d.a&& c.b==d.b);
 }

```

    }
    D. int operator+(node c,node d){
        node p;
        p.a=c.a+d.a;p.b=c.b+d.b;
        return p;
    }

```

13. 将 16 进制数 541BABA 转化为八进制数是()。
- A. 520335271 B. 520335272 C. 520345272 D. 520345271
14. 一开始有一个装了 N 个球的盒子,接下来重复以下操作:增加一个盒子并从其他盒子里各拿一个球加入新的盒子里,接着移去所有没有球的盒子,然后按球的个数递增顺序排列所有盒子。已知经过若干轮后,球的分布情况保持不变,则 N 的值可能是()。
- A. 7 B. 8 C. 9 D. 10
15. 世界上第一台通用计算机 ENIAC 于哪一年诞生?()。
- A. 1943 B. 1946 C. 1963 D. 1966

二、阅读程序(程序输入不超过数组或字符串定义的范围,判断题正确的填√,错误的填×;除特殊说明外,判断题 2 分,选择题 3 分,共计 40 分)

1.

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 struct node{
04     int frt, nxt;
05 }t[100010];
06 int a[100010];
07 bool del[100010];
08 queue<pair<int,int>>q;
09 int main(){
10     int n, sum;cin>>n>>sum;
11     for(int i=1;i<=n;i++){
12         cin>>a[i];
13         t[i].frt=(i==1)? n: (i-1);
14         t[i].nxt=(i==n)? 1: (i+1);
15     }
16
17     for(int i=1;i<=n;i++){
18         if(a[i]==a[t[i].nxt] || a[i]+a[t[i].nxt]==sum){
19             q.push(make_pair(i, t[i].nxt));
20         }
21     }
22
23     int ans=0;
24     while(!q.empty()){
25         int x=q.front().first;int y=q.front().second;q.pop();
26         if(del[x] || del[y])continue;
27
28         ans++;
29         del[x]=del[y]=1;

```

```

30         if (t[x].frt==y) swap(x,y);
31
32         int n1=t[x].frt;int n2=t[y].nxt;
33         t[n1].nxt=n2;t[n2].frt=n1;
34         if (n1!=n2 && (a[n1]==a[n2] || a[n1]+a[n2]==sum)) {
35             q.push(make_pair(n1,n2));
36         }
36     }
37     cout<<ans<<endl;
38     return 0;
39 }

```

保证 $1 \leq n \leq 10^5, 1 \leq \text{sum}, a_i \leq 10^9$ 。

●判断题

- (1) 对于任意输入, 程序都不会陷入死循环。()
- (2) 程序运行完 30 行的片段后, 一定满足 $x+1=y$ 。()
- (3) 程序输出的值不会超过 $\frac{n}{2}$ 。()
- (4) 将 34 行的“ $n1!=n2$ ”改为“ $\text{ans} * 2+1!=n$ ”后, 程序运行结果不变。()

●单选题

- (5) 如果输入数据如下, 程序运行到 22 行时, 排在队列开头的二元组为()。

10 5

1 2 5 2 1 2 3 4 2 4

- A. (2,3) B. (4,1) C. (6,7) D. (10,1)

- (6) 如果输入数据如下, 那么程序的输出结果为()。

10 5

1 2 5 2 1 2 3 4 2 4

- A. 3 B. 4 C. 5 D. 6

2.

```

01 #include<bits/stdc++.h>
02
03 #define LL long long
04 #define ull unsigned long long
05 #define F(i, j, k) for (int i=j; i<=k; ++i)
06 #define DF(i, j, k) for (int i=j; i>=k; --i)
07
08 using namespace std;
09 const int N=410;
10
11 char c1[N][N], c2[N][N], c3[N][N];
12
13 inline void solve() {
14     int n, Z;
15     cin>>n;
16     cin>>Z;
17     Z=Z/25;
18     F(i, 1, n)

```

```

19      F(j, 1, n)
20          cin>>c1[i][j];
21      if (n*Z%4!=0) { puts("error"); return; }
22      F(i, 1, n)
23          F(j, 1, n)
24              F(k1, 1, Z)
25                  F(k2, 1, Z)
26                      c2[(i-1)*Z+k1][(j-1)*Z+k2]=c1[i][j];
27      F(i, 1, n*Z/4)
28          F(j, 1, n*Z/4) {
29              char ok=c2[(i-1)*4+1][(j-1)*4+1];
30              F(k1, 1, 4)
31                  F(k2, 1, 4)
32                      if (c2[(i-1)*4+k1][(j-1)*4+k2]!=ok) { puts("
error"); return; }
33              c3[i][j]=ok;
34          }
35      F(i, 1, n*Z/4) {
36          F(j, 1, n*Z/4)
37              putchar(c3[i][j]);
38          puts("");
39      }
40 }
41
42 int main() {
43     int T; cin>>T;
44     while (T--) solve();
45     return 0;
46 }

```

输入数据保证是一张 $n \times n$ 的仅由小写字母构成的矩阵。

n 小于等于 250, Z 只会是 100 到 200 中的 25 的倍数。

有 T 组测试数据。

● 判断题

- (1) 只要 Z 等于 100 或者 200, 程序必定不会输出 error。 ()
- (2) 如果 n 等于 250, 程序可能会访问到未定义的数组。 ()
- (3) 如果 n 小于等于 100, 程序一定不会访问到未定义的数组。 ()
- (4) 将 33 行的 $c3[i][j]$ 改成 $c2[i][j]$, 37 行的 $c3[i][j]$ 改成 $c2[i][j]$, 程序运行结果不会发生变化。 ()

● 单选题

- (5) 如果输入数据如下, 那么程序输出为 ()。

```

8 125
aaaabbbb
aaaabbbb
aaaabbbb
aaaabbbb
aaaaaaa

```

aaaaaaaa

baaaaaaaaa

A. error

B. aaaaabbbbb

C. aaaaabbbbb

D. aaaaabbbbb

aaaaabbbbb

aaaaabbbbb

aaaaabbbbb

aaaaabbbbb

aaaaabbbbb

aaaaabbbbb

aaaaabbbbb

aaaaabbbbb

aaaaabbbbb

aaaaabbbbb

aaaaabbbbb

aaaaabbbbb

aaaaaaaa

aaaaaaaa

aaaaaaaa

aaaaaaaa

aaaaaaaa

aaaaaaaa

aaaaaaaa

aaaaaaaa

aaaaaaaa

bbaaaaaaaaa

bbaaaaaaaaa

bbaaaaaaaaa

bbaaaaaaaaa

bbaaaaaaaaa

bbaaaaaaaaa

3.

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 int u[500010], v[500010];
04 long long w[500010];
05 int f[62][100010];
06
07 int root_seeking(int tp, int x) {
08     if (f[tp][x] == x) return x;
09     f[tp][x] = root_seeking(tp, f[tp][x]);
10     return f[tp][x];
11 }
12
13 struct wzy {
14     int nxt, vertice;
15 } edge[1000010];
16 int head[100010]; int len = 0;
17 void add_edge(int x, int y) {
18     edge[++len].nxt = head[x]; edge[len].vertice = y; head[x] = len;
19     return;
20 }
21
22 bool vis[100010]; int tot = 0; int col = 0;
23 void dfs(int x) {
24     f[tot][x] = col; vis[x] = 1;
25     for (int i = head[x]; i; i = edge[i].nxt) {
26         int nop = edge[i].vertice;
27         if (vis[nop]) continue;
28         dfs(nop);
29     }
30     return;
31 }
32
33 int main() {

```

```

34  int n, m, q; long long V; cin >> n >> m >> q >> V;
35  for (int i = 1; i <= m; i++) cin >> u[i] >> v[i] >> w[i];
36  for (int i = 0; i <= 61; i++) {
37      for (int j = 1; j <= n; j++) f[i][j] = j;
38  }
39  tot = 0; long long tmp = 0;
40  for (int i = 59; i >= 0; i--) {
41      if (!(V >> i) & 1) {
42          tmp |= (1LL << i);
43
44          len = 0;
45          for (int j = 1; j <= n; j++) head[j] = vis[j] = 0;
46          for (int j = 1; j <= m; j++) {
47              if ((w[j] & tmp) != tmp) continue;
48              add_edge(u[j], v[j]); add_edge(v[j], u[j]);
49          }
50          col = 0;
51          for (int j = 1; j <= n; j++) {
52              if (vis[j]) continue;
53              col++; dfs(j);
54          }
55
56          tmp ^= (1LL << i); tot++;
57      } else {
58          tmp |= (1LL << i);
59      }
60  }
61  len = 0;
62  for (int j = 1; j <= n; j++) head[j] = vis[j] = 0;
63  for (int j = 1; j <= m; j++) {
64      if ((w[j] & tmp) != tmp) continue;
65      add_edge(u[j], v[j]); add_edge(v[j], u[j]);
66  }
67  col = 0;
68  for (int j = 1; j <= n; j++) {
69      if (vis[j]) continue;
70      col++; dfs(j);
71  }
72
73  for (int i = 1; i <= q; i++) {
74      int ui, vi; cin >> ui >> vi;
75      bool flag = false;
76      for (int j = 0; j <= tot; j++) {
77          if (f[j][ui] == f[j][vi]) { flag = true; break; }
78      }
79      cout << ((flag == true) ? 1 : 0);

```

```

80     }
81     return 0;
82 }

```

保证 $1 \leq n \leq 10^5, 1 \leq m, q \leq 5 \times 10^5, 1 \leq u_i, v_i \leq n, 0 < w_i, V < 2^{60}$ 。

● 判断题

- (1) 将 36 行的 61 改为 62, 程序能正常运行。()
 (2) 当程序在执行 44 行到 54 行的片段时, tmp 的值始终大于 V。()
 (3) 将 50 行或 67 行的“col=0;”删去, 程序输出结果均不变。()

● 单选题

- (4) 程序运行过程中 tot 能达到的最大值为()。
 A. 58 B. 59 C. 60 D. 61
 (5) 程序运行的理论复杂度是()。
 A. $O((m+q) \log V)$ B. $O((m+q) \log V \log n)$
 C. $O(m \alpha(n) \log V + q \log V)$ D. $O((m+q) \alpha(n) \log V)$
 (6) 对于如下输入数据, 程序的输出是()。

```

9 8 4 5
1 2 8
1 3 7
2 4 1
3 4 14
2 5 9
4 5 7
5 6 6
3 7 15
1 6
2 7
7 6
1 8

```

- A. 0110 B. 1010 C. 1000 D. 1011

三、完善程序(单选题, 每题 3 分, 共计 30 分)

1. 匹配

给定整数序列 $\{a_n\}$, 如果满足 $i-j=a_i-a_j$, 则在图上点 i 和点 j 之间连一条权值为 (a_i+a_j) 的无向边。求得到的无向图的最大权匹配的值。

其中 $n \leq 10^5, |a_i| \leq 10^9$ 。

试补全以下程序。

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 int a[100010], b[100010];
04 vector<int>v[100010];
05 int main() {
06     int T; cin>>T;
07     while(T--){
08         int n; cin>>n;
09         for(int i=1; i<=n; i++) ①;
10         for(int i=1; i<=n; i++){
11             cin>>a[i];

```

```

12         b[i]=a[i]-i;
13     }
14     sort(b+1,b+n+1);
15     int len= ② ;
16
17     for(int i=1;i<=n;i++){
18         int x=lower_bound(b+1,b+len+1,a[i]-i)-b;
19         v[x].push_back(a[i]);
20     }
21
22     long long ans=0;
23     for(int i=1;i<=len;i++){
24         int cnt=0;long long sum=0;
25         for(int j=0;j<(int)v[i].size();j++){
26             if(v[i][j]<0)continue;
27             cnt++;sum+=v[i][j];
28         }
29         if( ③ ){ans+=sum;continue;}
30
31         sort(v[i].begin(),v[i].end());
32         int tot=(int)v[i].size();
33         if( ④ ){
34             ans+=sum-v[i][0];
35         }else if( ⑤ ){
36             ans+=sum+v[i][tot-cnt-1];
37         }else{
38             ans+=sum-v[i][tot-cnt];
39         }
40     }
41     cout<<ans<<endl;
42 }
43 return 0;

```

(1)①处应填()。

- A. a[i]=0 B. b[i]=0 C. v[i]=0 D. v[i].clear()

(2)②处应填()。

- A. unique(b+1,b+n)-b-1 B. unique(b+1,b+n)-b
C. unique(b+1,b+n+1)-b-1 D. unique(b+1,b+n+1)-b

(3)③处应填()。

- A. cnt%2==0 B. cnt%2==1
C. (int)v[i].size()%2==0 D. (int)v[i].size()%2==1

(4)④处应填()。

- A. cnt%2==0 B. cnt%2==1 C. cnt==0 D. cnt==tot

(5)⑤处应填()。

- A. v[i][tot-cnt]+v[i][tot-cnt-1]>0
B. v[i][tot-cnt]+v[i][tot-cnt-1]<0
C. sum+v[i][tot-cnt-1]>0

D. $\text{sum} + v[i][\text{tot} - \text{cnt} - 1] < 0$

2. 反恐精英

给定 n 个点 m 条边的无向图, 有 k 个点被埋了炸弹, 你有一个优秀干员, 他可以切断任意一个点, 你需要雇佣 g 个普通干员, 他们只能切断前 g 个有炸弹的点, 求使得任意未被切断的有炸弹的点不连通的 g 的最小值。

n, m, k 小于等于 200000。

```
01 #include<bits/stdc++.h>
02
03 #define F(i, j, k) for (int i=j; i<=k; ++i)
04
05 using namespace std;
06
07 template <typename T> inline void checkmin(T &a, T b) { a=a<b ? a :
b; }
08
09 const int N=2e5+100;
10
11 int n, m, k, cnt, head[N], to[N<<1], nxt[N<<1], v[N], h[N], del[N],
    vis[N], depth[N], mi[N], sum[N], father[N], hao;
12
13 inline void addedge(int x, int y) {
14     nxt[++cnt]=head[x];
15     to[cnt]=y;
16     head[x]=cnt;
17 }
18
19 inline void dfs(int x) {
20     vis[x]=1;
21     sum[x]+=v[x];
22     for (int i=head[x]; i; i=nxt[i]) {
23         if (del[to[i]]) continue;
24         if (vis[to[i]]) {
25             checkmin(mi[x], ②);
26             continue;
27         }
28         father[to[i]]=x;
29         depth[to[i]]=depth[x]+1;
30         dfs(to[i]);
31         sum[x]+=sum[to[i]];
32     }
33 }
34
35 inline void doit(int x, int cur) {
36     int oknow=1, okk=0;
37     for (int i=head[x]; i; i=nxt[i]) {
38         if (del[to[i]]) continue;
```

```

39         if (father[to[i]] != x) continue;
40         if (sum[to[i]] > 1 || mi[to[i]] < depth[x]) {
41             if (sum[to[i]] == 1 && mi[to[i]] < depth[x]) {
42                 if (cur - sum[x] == 0 && okk == 0) {
43                     okk = 1;
44                     continue;
45                 }
46             }
47             ③
48             break;
49         }
50     }
51     if ( ④ ) oknow = 0;
52     if (oknow) hao = 1;
53     for (int i = head[x]; i; i = nxt[i]) {
54         if (del[to[i]]) continue;
55         if (father[to[i]] != x) continue;
56         doit(to[i], cur);
57     }
58 }
59
60 inline bool check(int x) {
61     F(i, 1, n) { del[i] = vis[i] = depth[i] = sum[i] = father[i] = 0; mi
[i] = n + 1; }
62     F(i, 1, x) del[h[i]] = 1;
63     int ok = 0;
64     F(i, 1, n)
65         if ( ① ) {
66             dfs(i);
67             int cur = sum[i];
68             if (ok == 1 && cur > 1) return 0;
69             if (cur > 1) {
70                 hao = 0;
71                 doit(i, cur);
72                 if (!hao) return 0;
73             }
74             if ( ⑤ ) ok = 1;
75         }
76     return 1;
77 }
78
79 int main() {
80     scanf("%d%d%d", &n, &m, &k);
81     cnt = 0;
82     F(i, 1, n) head[i] = v[i] = 0;
83     F(i, 1, m) {

```

```

84      int x, y;
85      scanf("%d%d%d", &x, &y);
86      addedge(x, y);
87      addedge(y, x);
88  }
89  F(i, 1, k) { scanf("%d", h + i); v[h[i]] = 1; }
90  int l = 0, r = k, ans = k;
91  while (l <= r) {
92      int mid = (l + r) >> 1;
93      if (check(mid)) { r = mid - 1; ans = mid; }
94      else l = mid + 1;
95  }
96  printf("%d\n", ans);
97  return 0;

```

(1) ①处应填()。

A. !del[i]

B. !vis[i]

C. !del[i] && !vis[i]

D. !del[i] || !vis[i]

(2) ②处应填()。

A. depth[to[i]]

B. depth[x]

C. mi[to[i]]

D. mi[x]

(3) ③处应填()。

A. hao = 0;

B. okk = 0;

C. oknow = 0;

D. cur = 0;

(4) ④处应填()。

A. cur-sum[x] > 1 || (cur-sum[x] == 0 && okk == 1)

B. cur-sum[x] > 1 || (cur-sum[x] == 1 && okk == 1)

C. cur-sum[x] > 0 || (cur-sum[x] == 0 && okk == 1)

D. cur-sum[x] > 0 || (cur-sum[x] == 1 && okk == 1)

(5) ⑤处应填()。

A. cur == 1

B. hao == 0

C. hao == 1

D. cur > 1